

# Orbit Academy — System Administrator's Guide

v5.5

## Contents

|                                                 |   |
|-------------------------------------------------|---|
| 2. Download & install . . . . .                 | 1 |
| 3. Ways to run it . . . . .                     | 3 |
| 4. The data file (server mode) . . . . .        | 5 |
| 7. Instructor dashboard (server mode) . . . . . | 5 |
| 8. Updating & testing . . . . .                 | 6 |
| 9. Troubleshooting . . . . .                    | 7 |

This guide collects the system-administration material for Orbit Academy: getting the files, the two run modes, hosting for a classroom, where the data lives and what to back up, account administration, updating, and troubleshooting. It is extracted from the Instructor's Guide (which adds the module-by-module teaching notes) and the Security Notes (which add the full per-release audit log); both ship alongside this file.

## 2. Download & install

Everything is plain HTML/JavaScript plus one small Node.js server file. There is **nothing to compile and no packages to install** — ever. The one third-party component (the Three.js 3-D library) is held **inside the course folder**, so a running install makes no internet requests at all. Fetch it once with `bash tools/fetch-vendor.sh` (see §2.3); after that the course is fully self-contained and suitable for air-gapped machines.

### 2.1 Get the files

**Option A — download a ZIP (no tools or GitHub knowledge needed):** 1. In a web browser, go to <https://github.com/JASONSS26/orbit-academy> (sign in to GitHub if it asks — the project is private, so use the account that was invited to it). 2. Click the green **<> Code** button near the top-right of the file listing, then click **Download ZIP** in the menu that appears. The file `orbit-academy-main.zip` lands in your Downloads folder. 3. Unzip it — Windows: right-click → **Extract All...**; macOS: double-click. Inside the resulting folder is an **academy** folder: that's the whole course. Move it anywhere you like.

**Option B — clone with git (if you have git):**

```
git clone https://github.com/JASONSS26/orbit-academy.git
cd orbit-academy/academy
```

## 2.2 Confirm the assets are intact (optional, no internet)

**There is nothing to download.** Three.js and all four planet maps are committed to the repository, so the clone you just made is already a complete, runnable, offline course. This is deliberate: it removes the install step, and it removes version drift — every classroom runs identical bytes.

After a ZIP round-trip or a USB copy it is still worth confirming nothing was lost:

```
cd path/to/academy
bash tools/fetch-vendor.sh --check
```

It recomputes the SHA-384 of every vendored file and compares it with the table in `public/vendor/NOTICE.md`, which also records where each file came from and under what license. Expected tail: `PASS` — every vendored asset matches `public/vendor/NOTICE.md`. On Windows use Git Bash or WSL (`README.md` has a pure-PowerShell equivalent). See `docs/MAINTENANCE.md` §2.5 for the full asset architecture.

If it reports `MISSING` or `MISMATCH`, restore with `git checkout -- public/vendor` or re-copy the folder. A missing library also produces an explicit on-screen banner rather than a blank canvas.

## 2.3 Install Node.js (only needed to run the tracked/server mode)

**What is Node.js and why do I need it?** The course’s account/roster server (`server.js`) is a JavaScript program, and Node.js is the (free, ~50 MB) program that runs it — the same way you’d need Python installed to run a `.py` file. Installing it changes nothing else on the computer. You do **not** need it at all for the standalone (no-login) mode — see §3, Options 0 and 1.

**Windows:** 1. Go to <https://nodejs.org> and download the green **LTS** installer (a `.msi` file). 2. Run it and click through (Next → Next → Install). All defaults are fine; you do **not** need the “tools for native modules” checkbox. 3. Verify: open a **new** Command Prompt (press the Windows key, type `cmd`, press Enter) and type `node --version`. If it prints a version like `v22.x`, you’re set. - If it says ‘node’ is not recognized, the window was opened before the install finished — close it and open a fresh one (the `PATH` only updates for newly opened windows).

**macOS:** 1. Go to <https://nodejs.org> and download the **LTS** installer (a `.pkg` file), then run it. (Homebrew users: `brew install node` works too.) 2. Verify: open Terminal (⌘-space, type “Terminal”, Enter) and type `node --version`.

Any recent version works — the server uses only Node’s built-ins, nothing to `npm install`, ever.

**Does this run on Windows?** Yes — the whole course (server included) is plain Node.js with no OS-specific code, and runs identically on Windows, macOS, and Linux. The one exception is the *developer* test suite (`test/run.sh`), which is a bash script — on Windows run it from Git Bash or WSL. Instructors and students never need it.

### 3. Ways to run it

Choose based on whether you need **central accounts and a roster** — and note **Option 0** if the machines have no internet (the course is designed for that case and is verified against it).

#### How the pieces fit together

*[Diagram — see the online version of this document.]*

Use this to decide what you need before installing anything: the left-hand path needs nothing at all, the right-hand path needs Node.js and gives you the tools row.

#### Option 0 — Standalone / air-gapped (no internet, ever)

**This is the default posture, and the one to use on a closed network.** Orbit Academy makes **zero outbound network calls** when it runs: the 3-D library is loaded from inside the course folder, the planet maps resolve locally, and nothing phones home. There is no npm install, no build step, and no telemetry of any kind.

**There is no preparation step.** Every byte the course needs — the 3-D library included — is committed to the repository. Move the whole academy folder to the target machine: copy it, zip it, clone it, or put it on a USB stick. Nothing has to be fetched first, on either end, and the target machine can be one that has never touched a network. On the target:

- **No-login use:** open `academy/public/gallery.html` in a browser. Done. (Start at *gallery*, not `index.html` — the hub is the server-mode page and needs `server.js`; it will say so and redirect you. Note also that from a `file://` URL Module 8's forward window shows flat-shaded planets, since browsers block pixel reads on local images; it states this on screen and nothing else changes.)
- **Tracked cohort:** `node server.js`, then browse to `http://localhost:8080` (see Option 2).

Verify the posture at any time — both commands are safe to run on the closed network:

```
bash tools/fetch-vendor.sh --check      # what is present; whether anything could call out
node test/no-external-calls.test.js     # PROVES no outbound calls (static scan + live m
```

The second one is the important one for an accreditation conversation: it scans every shipped file for anything a browser would fetch, *and* separately executes all eight simulators against instrumented network primitives (`fetch`, `XMLHttpRequest`, `WebSocket`, `EventSource`, `sendBeacon`, `image` and `script src`) and fails if a single absolute URL is attempted. It should print `PASSED` — the course makes no outbound network calls. Air-gap clean.

Two honest caveats, neither of which sends anything anywhere:

- The worksheets carry ~99 **further-reading links** (Wikipedia, NASA, CelesTrak). They are ordinary links: nothing is fetched unless a student clicks one, and on a closed network they simply fail to open. They are listed in each worksheet's *Further reading* panel if you would rather point students at an internal mirror.
- The Earth and Moon maps fall back to **schematic** versions that ship in the repo (ocean blue with a 15-degree graticule, marked equator and tropics). If `fetch-vendor.sh` also managed to pull the photographic maps, those are used instead. Both

are local; the difference is cosmetic, and the schematic grid is arguably better for teaching rotation and inclination.

### Option 1 — Standalone (zero install, no accounts)

The worksheets and simulators are static files. If no backend is present, each worksheet automatically runs in **standalone mode**: it renders normally and saves the student's progress in that browser's own storage (localStorage). No logins, no central roster — the course just works from a local file or any static host.

- **Run it with no server at all:** open `academy/public/gallery.html` directly in a browser, or serve the folder with any static server (e.g. `python3 -m http.server` from inside `academy/public/`).
- **Trade-off:** progress lives in each browser, so it doesn't follow a student between devices, and there's no roster. Best for demos, self-study, and quick reviewer previews.

A small badge ("● standalone — saved in this browser") appears on the worksheet in this mode, so students know their progress is local.

□ **A note on GitHub Pages.** It's tempting to host the static site on GitHub Pages, but on standard GitHub a **Pages site is readable by anyone with the URL even when the repo is private** — there is no access control. Access-restricted Pages exists only on GitHub Enterprise Cloud. So do **not** use Pages if the material must stay limited to authorized people. For an authorized-only preview, have reviewers **clone the repo and run it locally** (they already have repo access), or use **Codespaces** (private port-forwarding), or a host behind your own SSO.

### Option 2 — Full server (accounts, saved progress, instructor roster)

Run the Node server for a managed cohort with central accounts and a dashboard.

**Easiest — no terminal needed:** in the `academy` folder, double-click the launcher: - **Windows:** `start-academy.bat` - **macOS:** `start-academy.command` (first time, macOS may balk: right-click → **Open** → **Open**)

The launcher checks that Node.js is installed (and sends you to `nodejs.org` if it isn't), starts the server, and opens your browser to it. **Keep its window open** — closing it stops the server.

**Or from a terminal:**

```
cd path/to/academy          # tip: type "cd " then DRAG the academy folder onto the window
node server.js              # serves http://localhost:8080
```

Optional settings (note the syntax differs by shell):

*# macOS / Linux:*

```
PORT=9000 node server.js
```

```
ORBIT_DATA=/path/to/academy_data.json node server.js
```

*# Windows Command Prompt:*

```
set PORT=9000 && node server.js
```

```
# Windows PowerShell:
$env:PORT=9000; node server.js
```

- Open **http://localhost:8080**. **The first account you register becomes the instructor.** Register yours first, then have students register their own.
- Progress and prerequisite gating are enforced server-side; the instructor account sees a roster of all students and their module/pass status.
- For a classroom, run it on one machine on the LAN and give students that machine's address (e.g. `http://10.0.0.5:8080`). For internet-facing use, put it behind HTTPS + a reverse proxy with rate-limiting (see `docs/SECURITY.md`).

### Option 3 — Both

Publish the standalone build on GitHub Pages for broad/self-study access, **and** run the server for tracked cohorts. They share the same files.

|                     | Standalone (Pages)             | Full server                       |
|---------------------|--------------------------------|-----------------------------------|
| Install             | none                           | Node.js                           |
| Accounts / login    | no                             | yes (first user = instructor)     |
| Progress saved      | per-browser (localStorage)     | per-account (follows the student) |
| Prerequisite gating | open (all modules available)   | enforced server-side              |
| Instructor roster   | no                             | yes                               |
| Best for            | open access, demos, self-study | managed training cohorts          |

## 4. The data file (server mode)

- `academy_data.json` holds all accounts (passwords are **scrypt-hashed**, never plain-text) and progress. It is created automatically on first run.
- It is **gitignored and must never be committed** — it contains password hashes. The repo also ignores its rolling backup (`.bak`) and temp (`.tmp`) files.
- The server writes atomically (temp file + rename) and keeps a rolling `.bak`, so an accidental crash or delete won't lose accounts. To relocate it, set the `ORBIT_DATA` environment variable.
- **Back it up** by copying `academy_data.json` somewhere safe periodically.

## 7. Instructor dashboard (server mode)

Log in with the instructor account (the first one registered). The hub shows a **roster**: each student, which modules they've completed, and their pass status. Prerequisite gating means a student can't skip ahead — the server enforces it regardless of what the browser does.

To reset a cohort, stop the server and archive/remove `academy_data.json` (keep a backup first); a fresh file is created on next start, and the first new registration becomes the instructor again.

## 7.1 The worksheet editor — change the course content without touching code

The instructor dashboard has an `✎ Edit worksheets` button (it opens `editor.html`; instructor accounts only — students are refused by the server, not just the page). It lets you rewrite any worksheet — objectives, tutorial text, every exercise’s teach/predict/do/observe/think blocks, the quiz questions, options, correct answers and per-option feedback, and the final-check questions — from a form, with no code editing.

What you need to know before using it (all verified against the running server):

- **What it edits:** the *published* workbook set in `workbooks/active/` — the files the server actually serves to students. Saves take effect on each student’s **next worksheet load** (the pages are served no-cache, so a browser refresh picks the edit up).
- **It’s hard to break things.** Every save is first executed in a sandbox and must produce a valid worksheet — a save that would white-screen a worksheet is **rejected** with an error and the old version stays live. Writes are atomic, and every successful save first snapshots the outgoing version as a timestamped `worksheet<N>.data.<date>.bak` beside it, so any edit can be undone by renaming the `.bak` back over the `.data.js` file.
- **One caveat — standalone mode:** the zero-install/no-server mode serves its *own* copies of the worksheets (in `public/`). Editor saves do **not** touch those. If you use standalone mode and want your edits there too, copy the files across after editing: `cp workbooks/active/worksheet*.data.js public/` (and note the copies lose the source files’ code comments — the content itself round-trips exactly).
- **Versioned sets:** sibling folders under `workbooks/` (e.g. `workbooks/2026-spring/`) hold inactive versions; “publishing” a set is just renaming folders so yours is called `active`. Works the same on Windows and macOS.

A sensible workflow: keep the class on `active`, clone it to a dated folder before a big revision (`cp -r workbooks/active workbooks/2026-fall-draft`), edit live, and if a revision goes sideways, swap the folder names back.

---

## 8. Updating & testing

- Pull the latest files (ZIP or `git pull`).
- If you run the server, sanity-check it after updating:

```
cd academy
```

```
bash test/run.sh          # functional + security + DoS checks, on an isolated temp data
```

The tests never touch your real `academy_data.json`.

- See `docs/CHANGELOG.md` for what changed, and `docs/SECURITY.md` for the per-release audit log.

## 9. Troubleshooting

- **“Please sign in first.”** — You’re in server mode but not logged in. Register/log in at the hub. (If you *want* no-login use, host the static files without the server — §3 Option 1.)
  - **The worksheet window didn’t open.** — The browser blocked the pop-up. Allow pop-ups for the site, then click “Re-open the worksheet.”
  - **3-D view is blank, or a banner says “3-D library not loaded”.** — `public/vendor/three.min.js` is missing from the course folder; it ships in the repository, so this means an incomplete copy or extraction, **not** a missing internet connection (the course never needs one). Run `bash tools/fetch-vendor.sh --check`, or restore with `git checkout -- public/vendor`. A missing *texture* is harmless by comparison — the sim falls back to the schematic map that also ships in the repo.
  - **Progress didn’t save across devices.** — That’s standalone mode (per-browser storage). Use the server (Option 2) for progress that follows the student.
  - **Lost accounts.** — Restore `academy_data.json` from its `.bak` (same folder) or your backup.
-